

M2 - Sistemi operativi e linguaggi

Corso: CyberSecurity

Studente: Martina Foresta

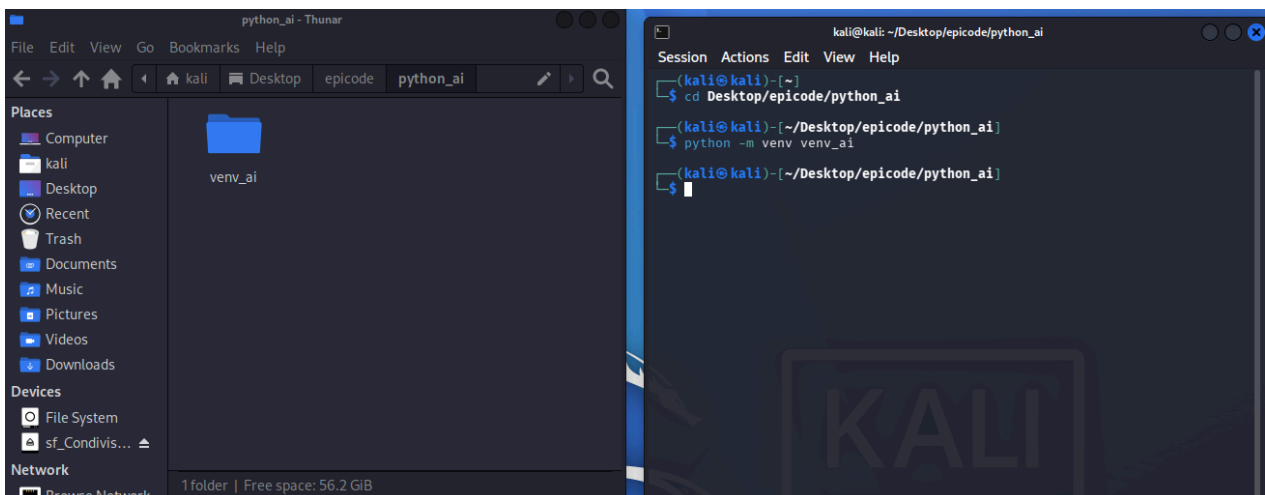
Esercizio: Setup Gemini API su Kali Linux

Traccia: creare un ambiente virtuale Python in Kali Linux, installare le librerie necessarie per Gemini, impostare una API key e realizzare un primo script che invii una domanda al modello e stampi la risposta.

Per svolgere il laboratorio ho usato Kali Linux nella macchina virtuale Oracle VirtualBox.

Iniziamo aprendo il terminale e spostandoci nella cartella del progetto

~/Desktop/epicode/python_ai. Qui ho creato l'ambiente virtuale con il comando `python -m venv venv_ai`. L'ambiente virtuale serve a tenere separate le librerie del progetto da quelle già presenti nel sistema.



Dalla schermata possiamo vedere che la cartella `venv_ai` è stata creata correttamente all'interno della directory di lavoro.

Ora attiviamo l'ambiente virtuale con il comando `source venv_ai/bin/activate`. Quando davanti al prompt compare `(venv_ai)`, significa che i comandi Python e pip useranno l'ambiente appena creato.

```
(kali㉿kali)-[~/Desktop/epicode/python_ai]
$ source venv_ai/bin/activate

(venv_ai)-(kali㉿kali)-[~/Desktop/epicode/python_ai]
$ pip install google-genai pandas numpy
```

Successivamente ho installato le librerie con `pip install google-genai pandas numpy`. La libreria `google-genai` permette allo script di comunicare con Gemini; `pandas` e `numpy` sono librerie utili per lavorare con dati e valori numerici.

```
Collecting urllib3<3, ≥1.26 (from requests<3.0.0, ≥2.28.1→google-genai)
  Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting python-dateutil≥2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py3-none-any.whl.metadata (8.4 kB)
Collecting cffi≥2.0.0 (from cryptography≥41.0.5→google-auth<3.0.0, ≥2.56.0→google-auth[requests]<3.0.0, ≥2.56.0→google-genai)
  Downloading cffi-2.1.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.5 kB)
Collecting pycparser (from cffi≥2.0.0→cryptography≥41.0.5→google-auth<3.0.0, ≥2.56.0→google-auth[requests]<3.0.0, ≥2.56.0→google-genai)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Collecting pyasn1<0.7.0, ≥0.6.1 (from pyasn1-modules≥0.2.1→google-auth<3.0.0, ≥2.56.0→google-auth[requests]<3.0.0, ≥2.56.0→google-genai)
  Downloading pyasn1-0.6.4-py3-none-any.whl.metadata (8.4 kB)
Collecting six≥1.5 (from python-dateutil≥2.8.2→pandas)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading google_genai-2.20.0-py3-none-any.whl (1.1 MB)
  1.1/1.1 MB 32.6 MB/s 0:00:00
Downloading anyio-4.14.2-py3-none-any.whl (125 kB)
Downloading distro-1.9.0-py3-none-any.whl (20 kB)
Downloading google_auth-2.57.0-py3-none-any.whl (259 kB)
Downloading httpx-0.28.1-py3-none-any.whl (73 kB)
Downloading httcore-1.0.9-py3-none-any.whl (78 kB)
Downloading pydantic-2.13.5-py3-none-any.whl (472 kB)
Downloading pydantic_core-2.46.5-cp314-cp314-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.1 MB)
  2.1/2.1 MB 80.4 MB/s 0:00:00
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.5.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (251 kB)
Downloading idna-3.19-py3-none-any.whl (68 kB)
Downloading tenacity-9.1.4-py3-none-any.whl (28 kB)
Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Downloading websockets-16.1.1-cp314-cp314-manylinux_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (187 kB)
Downloading pandas-3.0.5-cp314-cp314-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (11.0 MB)
  11.0/11.0 MB 48.0 MB/s 0:00:00
Downloading numpy-2.5.2-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.7 MB)
  16.7/16.7 MB 35.3 MB/s 0:00:00
Downloading annotated_types-0.8.0-py3-none-any.whl (13 kB)
Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
Downloading cryptography-50.0.1-cp311-abis-manylinux_2_34_x86_64.whl (4.7 MB)
  4.7/4.7 MB 27.1 MB/s 0:00:00
Downloading cffi-2.1.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (221 kB)
Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Downloading pyasn1_modules-0.4.2-py3-none-any.whl (181 kB)
Downloading pyasn1-0.6.4-py3-none-any.whl (84 kB)
Downloading python_dateutil-2.9.0.post0-py3-none-any.whl (229 kB)
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Downloading sniffio-1.3.1-py3-none-any.whl (10 kB)
Installing collected packages: websockets, urllib3, typing-extensions, tenacity, sniffio, six, pycparser, pyasn1, numpy, idna, h11, distro, charset_normalizer, certifi, annotated-types, typing-inspection, requests, python-dateutil, pydantic-core, pyasn1-modules, httcore, cffi, anyio, pydantic, pandas, httpx, cryptography, google-auth, google-genai
Successfully installed annotated-types-0.8.0 anyio-4.14.2 certifi-2026.7.22 cffi-2.1.1 charset_normalizer-3.5.1 cryptography-50.0.1 distro-1.9.0 google-auth-2.57.0 google-genai-2.20.0 h11-0.16.0 httcore-1.0.9 httpx-0.28.1 idna-3.19 num
py-2.5.2 pandas-3.0.5 pyasn1-0.6.4 pyasn1-modules-0.4.2 pycparser-3.0 pydantic-2.13.5 pydantic-core-2.46.5 python-dateutil-2.9.0.post0 requests-2.34.2 six-1.17.0 sniffio-1.3.1 tenacity-9.1.4 typing_extensions-4.16.0 typing-inspection-0.
4.4 urllib3-2.7.0 websockets-16.1.1
```

Durante l'installazione pip scarica i pacchetti richiesti e le loro dipendenze. Bisogna attendere la fine del caricamento senza chiudere il terminale.

Quando il processo è terminato, pip mostra la scritta *Successfully installed*. Questa conferma indica che le librerie sono state installate correttamente nel virtual environment.

```
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ pip install google-generativeai pandas numpy
Collecting google-generativeai
  Downloading google_generativeai-2.20.0-py3-none-any.whl.metadata (56 kB)
Collecting pandas
  Downloading pandas-3.0.5-cp314-cp314-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting numpy
  Downloading numpy-2.5.2-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting anyio<5.0.0, ≥4.8.0 (from google-generativeai)
  Downloading anyio-4.4.2-py3-none-any.whl.metadata (4.6 kB)
Collecting httpx<1.0.0, ≥0.28.1 (from google-generativeai)
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting google-auth<3.0.0, ≥2.56.0 (from google-generativeai)
  Downloading google_auth-2.57.0-py3-none-any.whl.metadata (6.0 kB)
Collecting requests<3.0.0, ≥2.12.5 (from google-generativeai)
  Downloading requests-2.32.4-py3-none-any.whl.metadata (4.6 kB)
Collecting pydantic<2.13.5, ≥2.12.5 (from google-generativeai)
  Downloading pydantic-2.12.5-py3-none-any.whl.metadata (110 kB)
Collecting tenacity<9.0.0, ≥8.2.3 (from google-generativeai)
  Downloading tenacity-9.1.4-py3-none-any.whl.metadata (1.2 kB)
Collecting websockets<17.0, ≥13.0.0 (from google-generativeai)
  Downloading websockets-16.1.1-cp314-cp314-manylinux1_x86_64.manylinux_2_5_x86_64.whl.metadata (6.8 kB)
Collecting typing_extensions<5.0.0, ≥4.14.0 (from google-generativeai)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting distro<2, ≥1.7.0 (from google-generativeai)
  Downloading distro-1.9.0-py3-none-any.whl.metadata (6.8 kB)
Collecting sniffio (from google-generativeai)
  Downloading sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)
Collecting idna<3.0, ≥4.8.0 (from google-generativeai)
  Downloading idna-3.19-py3-none-any.whl.metadata (9.2 kB)
Collecting httpcore<1.0.0, ≥0.28.1 (from google-generativeai)
  Downloading httpcore-1.0.7-py3-none-any.whl.metadata (2.1 kB)
Collecting pyasn1-modules<0.4.2, ≥0.0.8 (from google-generativeai)
  Downloading pyasn1_modules-0.4.2-py3-none-any.whl.metadata (3.5 kB)
Collecting cryptography<45.0.0, ≥45.0.0 (from google-generativeai)
  Downloading cryptography-45.0.1-cp311-cp313-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting certifi (from google-generativeai)
  Downloading certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
Collecting httpcore<1.0.0, ≥0.28.1 (from google-generativeai)
  Downloading httpcore-1.0.7-py3-none-any.whl.metadata (2.1 kB)
Collecting h11<0.16.0, ≥0.16.0 (from google-generativeai)
  Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting annotated-types<0.6.0, ≥0.6.0 (from google-generativeai)
  Downloading annotated_types-0.6.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core<2.46.5, ≥2.46.5 (from google-generativeai)
  Downloading pydantic_core-2.46.5-cp314-cp314-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.6 kB)
Collecting typing-inspection<0.4.2, ≥0.4.2 (from google-generativeai)
  Downloading typing_inspection-0.4.4-py3-none-any.whl.metadata (2.6 kB)
Collecting charset-normalizer<4, ≥2 (from google-generativeai)
  Downloading charset_normalizer-3.5.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (45 kB)
Collecting urllib3<2.2.0, ≥2.2.0 (from google-generativeai)
  Downloading urllib3-2.2.7-py3-none-any.whl.metadata (6.9 kB)
```

Per salvare le versioni dei pacchetti installati ho usato il comando `pip freeze >> requirements.txt`. Questo comando crea il file `requirements.txt`, utile se in futuro voglio ricreare lo stesso ambiente.

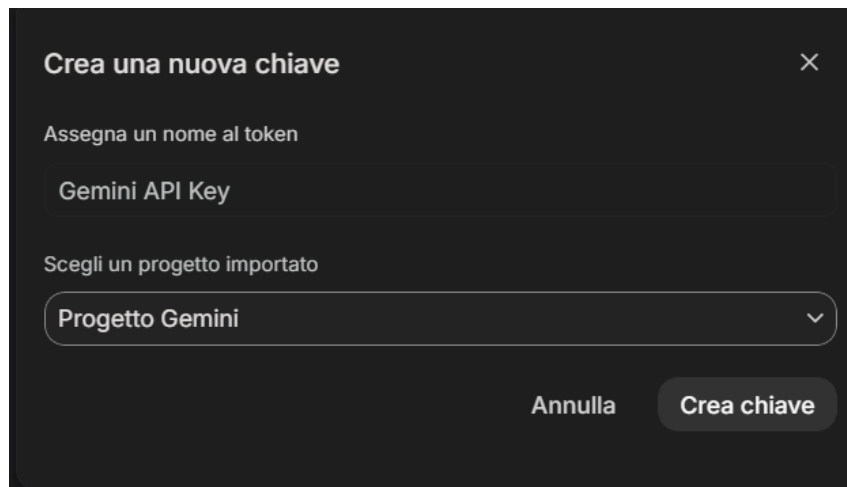
```
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ pip freeze >> requirements.txt

(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ ls
requirements.txt  venv_ai

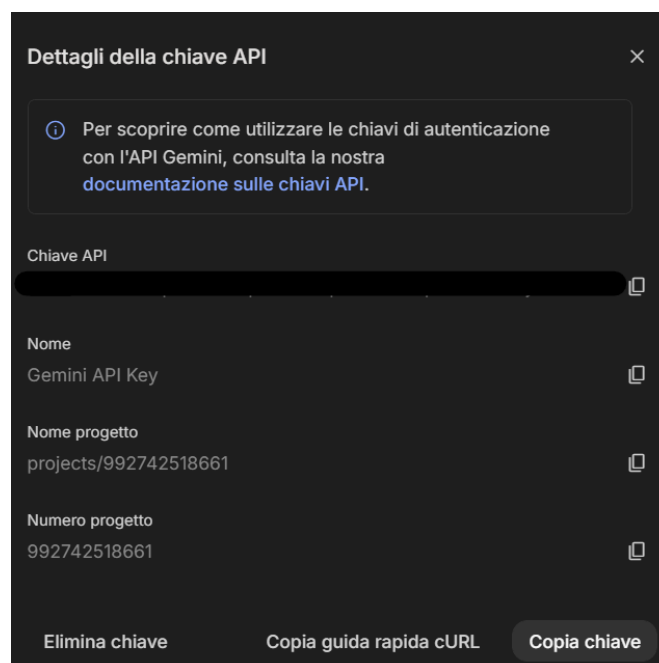
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$
```

Con il comando `ls` ho verificato che nella cartella sono presenti sia `venv_ai` sia il file `requirements.txt`.

Per usare Gemini serve una API key. Dal servizio Google ho creato una nuova chiave, le ho assegnato il nome Gemini API Key e l'ho collegata al progetto Progetto Gemini.



La chiave API è personale e non deve essere pubblicata. Per questo nelle schermate il suo valore è stato oscurato. Invece di inserirla nel file Python, ho scelto di salvarla in una variabile d'ambiente.

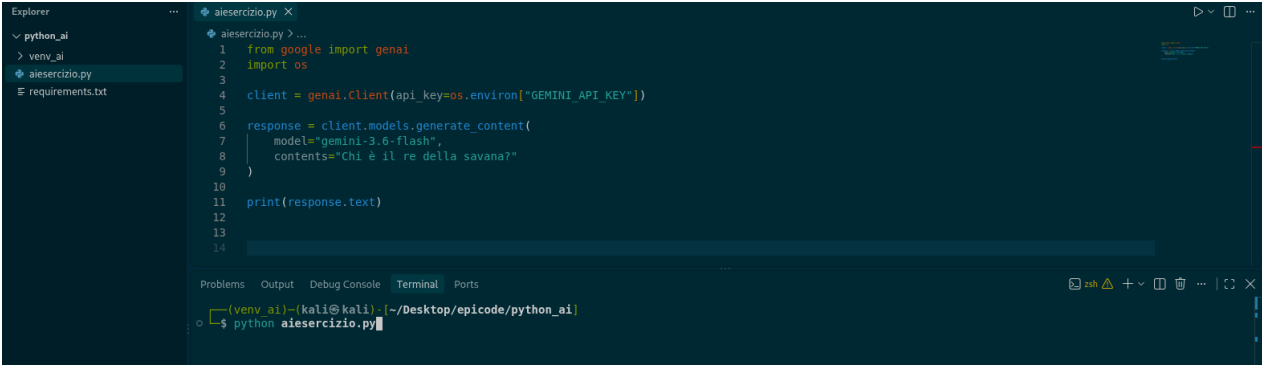


```
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ export GEMINI_API_KEY=[REDACTED]

(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ echo export GEMINI_API_KEY=\"${GEMINI_API_KEY}\" >> ~/.zshrc
```

Con il comando `export GEMINI_API_KEY=...` ho impostato la variabile per la sessione corrente. Ho poi aggiunto lo stesso valore al file `~/.zshrc`, così la variabile rimane disponibile anche quando riapro il terminale.

A questo punto ho creato il file `aiesercizio.py` in Visual Studio Code. Nella prima riga importo `genai`, cioè la libreria per Gemini, mentre con `import os` posso leggere la variabile d'ambiente che contiene la API key.



```
1 from google import genai
2 import os
3
4 client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
5
6 response = client.models.generate_content(
7     model="gemini-3.6-flash",
8     contents="Chi è il re della savana?"
9 )
10
11 print(response.text)
12
13
14
```

Problems Output Debug Console Terminal Ports

(venv_ai)-(kali@kali) - [~/Desktop/epicode/python_ai]
\$ python aiesercizio.py

Con `genai.Client(api_key=os.environ["GEMINI_API_KEY"])` creo il client usando la chiave salvata nell'ambiente. Poi `generate_content` invia al modello `gemini-3.6-flash` la domanda: `Chi è il re della savana?`. Infine `print(response.text)` stampa il testo ricevuto.

Per eseguire il programma ho usato il comando `python aiesercizio.py`, sempre con l'ambiente virtuale attivo.

Dopo l'avvio dello script Gemini ha risposto che il re della savana è tradizionalmente il leone e ha spiegato il motivo facendo riferimento al suo aspetto, al comportamento da predatore e alla cultura popolare.

```
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
└─$ python aiesercizio.py
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
Il "re della savana" è tradizionalmente il leone.

Gli è stato dato questo titolo per diversi motivi:

1. Aspetto maestoso: La folta criniera del leone maschio ricorda una corona e gli dona un'aria regale.
2. Predatore alfa: Si trova in cima alla catena alimentare nel suo habitat naturale.
3. Forza e comportamento: Il suo potente ruggito (che si può sentire fino a 8 km di distanza) e la sua presenza dominante lo rendono il simbolo della forza.
4. Cultura popolare: Storie, favole e film (come il celebre capolavoro Disney Il Re Leone) hanno consolidato questo titolo nell'immaginario collettivo.

*Curiosità:* A volte viene chiamato erroneamente anche "re della giungla", ma in realtà i leoni vivono nelle savane e nelle praterie, non nelle foreste pluviali!

(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
└─$
```

In conclusione, l'esercizio è stato svolto correttamente. Ho creato e attivato l'ambiente virtuale, installato le librerie, salvato la API key in modo sicuro, creato il file requirements.txt e ottenuto una risposta da Gemini tramite il mio primo script Python.